

Documento A – Conceitos e Escopo de Teste

Projeto: Contador de Corte para a PackBag (PI - Grupo 8 – Semestre 3)

Tecnologia: Flutter/Dart + Firebase

Arquitetura: MVVM

Norma base: ISO/IEC/IEEE 29119-1

1. Item de Teste

O item de teste deste ciclo é composto pelos módulos de autenticação (cadastro e login), pelo serviço de processamento de dados de produção e, adicionalmente, pelo serviço de autenticação com Firebase (AuthServices). As classes sob teste são:

- LoginViewmodel
- RegisterViewmodel
- ContagemCortesService
- AuthServices (testes de integração com Firebase Authentication)

2. Requisitos

• RF01 — Cadastro de Usuário

O sistema deve permitir que novos usuários realizem cadastro informando nome, e-mail e senha para acesso ao aplicativo.

• RF02 — Login do Usuário

O sistema deve permitir que usuários cadastrados realizem login utilizando e-mail e senha para acessar o sistema.

• RF03 — Recuperação de Senha

O sistema deve permitir que o usuário solicite a redefinição de senha por meio do e-mail cadastrado.

• RF04 — Atualização de Perfil

O sistema deve permitir que o usuário altere suas informações pessoais, como nome, foto de perfil e demais dados cadastrados.

• RF05 — Cadastro de Máquinas

O sistema deve permitir o cadastro de máquinas de corte no sistema para monitoramento da produção.

- **RF06 — Associação de Máquina a Usuário**

O sistema deve permitir associar uma máquina cadastrada a um operador responsável.

- **RF07 — Visualização do Dashboard**

O sistema deve apresentar um dashboard com dados de produção capturados das máquinas cadastradas.

- **RF08 — Gráfico de Produção Diária**

O sistema deve apresentar gráficos que exibam a produção diária das máquinas monitoradas.

- **RF09 — Resumo de Produção com IA**

O sistema deve gerar automaticamente um resumo textual da produção utilizando inteligência artificial com base nos dados coletados.

3. Escopo

Serão testados:

3.1 LoginViewmodel (testes de unidade)

- Validação de e-mail (campo obrigatório, formato inválido, formato válido);
- Validação de senha (campo obrigatório, comprimento mínimo de 6 caracteres, formato válido);
- Atribuição de valores aos controladores de texto;
- Notificação de listeners na alternância de visibilidade da senha.

3.2 RegisterViewmodel (testes de unidade)

- Validação de nome (obrigatório, mínimo de 3 caracteres, válido);
- Validação de e-mail (obrigatório, formato inválido, válido);
- Validação de senha (obrigatório, mínimo de 6 caracteres, válido);
- Validação de confirmação de senha (obrigatória, coincidência com a senha);
- Alternância de visibilidade dos campos de senha e confirmação.

3.3 ContagemCortesService (testes de unidade)

- Criação de objeto LogPHR a partir de dados simulados do Firestore;
- Cálculo do total de peças (calcularTotalPecas), incluindo lista vazia;
- Cálculo do percentual da meta (calcularPercentualMeta), com limite de 100 %;

- Geração de pontos para gráfico (gerarSpotsDoGrafico) agrupados por hora, incluindo lista vazia.

3.4 AuthServices (testes de integração)

- Criação de conta com e-mail e senha (fluxo real no Firebase);
- Login com credenciais válidas após criação da conta;
- Tentativa de login com senha inválida (deve gerar exceção);
- Logout (usuário atual deve se tornar nulo);
- Solicitação de redefinição de senha (reset password).

4. Fora do Escopo

- Cadastro completo com interface de usuário (telas e widgets);
- Login social ou métodos alternativos de autenticação;
- Atualização de perfil de usuário (RF04);
- Cadastro de máquinas (RF05) e associação a operadores (RF06);
- Visualização do dashboard e gráficos em nível de interface;
- Resumo de produção com IA (RF09);
- Regras de negócio: inatividade (RN02), bloqueio após tentativas (RN03), verificação real de unicidade de e-mail (RN04) – não são alvo direto dos testes;
- Requisitos não funcionais: desempenho (RNF01), segurança de armazenamento (RNF02), compatibilidade (RNF03), formatação (RNF04);
- Testes de sistema ou aceitação;
- Testes em múltiplos dispositivos físicos.

5. Glossário

- **4.1 Item de Teste:** conjunto formado pelos ViewModels, Service e Serviço de autenticação.
- **4.2 Base de Teste:** requisitos funcionais e regras de negócio que dirigem os testes do ciclo.
- **4.3 Condição de Teste:** situação ou comportamento que precisa ser verificado.
- **4.4 Caso de Teste:** conjunto de entradas, passos e resultados esperados.
- **4.5 Teste de unidade:** teste isolado de uma classe (ViewModel ou Service), sem dependências externas reais.
- **4.6 Teste de integração:** teste que verifica a interação com componente externo real (Firebase Authentication).
- **4.7 Mock/Fake:** substituto controlado para dependências reais (não utilizado nos testes de integração).
- **4.8 Evidência:** registro do resultado da execução.
- **4.9 Service:** classe com lógica de negócio independente da interface.
- **4.10 Spot:** ponto no gráfico (eixo x = hora, eixo y = valor acumulado).

6. Base de Teste

6.1 Requisito Funcional RF01 – Cadastro de Usuário

O sistema deve permitir que novos usuários realizem cadastro informando nome, e-mail e senha.

Cobertura nos testes de unidade (RegisterViewmodel):

- **RF01.1** – Nome: rejeitar vazio, exigir mínimo de 3 caracteres, aceitar nome válido.
- **RF01.2** – E-mail: rejeitar vazio, rejeitar formato inválido, aceitar formato válido.
- **RF01.3** – Senha: rejeitar vazia, rejeitar menos de 6 caracteres, aceitar válida.
- **RF01.4** – Confirmação de senha: rejeitar vazia, rejeitar diferente da senha, aceitar igual.

Cobertura nos testes de integração (AuthServices):

- **RF01.5** – O serviço de autenticação deve criar uma conta no Firebase com e-mail e senha, retornando um usuário válido.

6.2 Requisito Funcional RF02 – Login do Usuário

O sistema deve permitir que usuários cadastrados realizem login utilizando e-mail e senha.

Cobertura nos testes de unidade (LoginViewmodel):

- **RF02.1** – E-mail: rejeitar vazio, rejeitar formato inválido, aceitar formato válido.
- **RF02.2** – Senha: rejeitar vazia, rejeitar menos de 6 caracteres, aceitar válida.
- **RF02.3** – Controladores de texto devem armazenar os valores informados.
- **RF02.4** – A visibilidade da senha deve alternar e notificar os observadores.

Cobertura nos testes de integração (AuthServices):

- **RF02.5** – Login com credenciais válidas deve retornar um usuário autenticado.
- **RF02.6** – Login com senha incorreta deve gerar uma exceção.
- **RF02.7** – Após logout, o usuário atual deve ser nulo (encerramento de sessão).

6.3 Requisito Funcional RF03 – Recuperação de Senha

O sistema deve permitir que o usuário solicite a redefinição de senha por meio do e-mail cadastrado.

Cobertura nos testes de integração (AuthServices):

- **RF03.1** – O método de reset de senha deve ser chamado para um e-mail existente sem lançar exceções (validação de integração).

6.4 Requisitos Relacionados ao Dashboard e Gráficos

- **RF07** – Visualização do Dashboard: o serviço ContagemCortesService deve processar dados de produção para alimentar o dashboard, realizando cálculos de total de peças e percentual de meta.
- **RF08** – Gráfico de Produção Diária: o serviço deve gerar pontos de dados (FISpot) agrupados por hora para exibição em gráficos de produção.

6.5 Requisito Funcional RF08 - Gráfico de Produção Diária

O sistema deve exibir corretamente gráficos que reproduzem a contagem diária exata de cortes proveniente do protótipo físico.

Cobertura nos testes de unidade (ContagemCortesService)

- RF08.1 - É retornado ao usuário os dados vindos do Firestore.
- RF08.2 - É exibido a data e os valores de acordo com o período solicitado

56.6 Regra de Negócio RN01 – Campos Obrigatórios

O sistema deve impedir o cadastro/login com campos obrigatórios não preenchidos. Os testes de unidade dos validadores (Login e Register) verificam que campos vazios são rejeitados com as mensagens apropriadas.

7. Tipos de Teste

- **Testes de unidade:** aplicados isoladamente às classes LoginViewmodel, RegisterViewmodel e ContagemCortesService.
- **Testes de integração:** aplicados à classe AuthServices, interagindo diretamente com o serviço Firebase Authentication.

8. Condições Iniciais de Teste

8.1 LoginViewmodel (testes de unidade)

- **CT01** – Rejeitar e-mail vazio (RF02.1).
- **CT02** – Rejeitar e-mail inválido (RF02.1).
- **CT03** – Aceitar e-mail válido (RF02.1).

- **CT04** – Rejeitar senha vazia (RF02.2).
- **CT05** – Rejeitar senha com menos de 6 caracteres (RF02.2).
- **CT06** – Aceitar senha válida (RF02.2).
- **CT07** – Controladores devem armazenar os valores (RF02.3).
- **CT08** – Alternância de visibilidade notifica listeners (RF02.4).

8.2 RegisterViewmodel (testes de unidade)

- **CT09** – Rejeitar nome vazio (RF01.1).
- **CT10** – Rejeitar nome com menos de 3 caracteres (RF01.1).
- **CT11** – Aceitar nome válido (RF01.1).
- **CT12** – Rejeitar e-mail vazio no cadastro (RF01.2).
- **CT13** – Rejeitar e-mail inválido no cadastro (RF01.2).
- **CT14** – Aceitar e-mail válido no cadastro (RF01.2).
- **CT15** – Rejeitar senha vazia no cadastro (RF01.3).
- **CT16** – Rejeitar senha curta no cadastro (RF01.3).
- **CT17** – Aceitar senha válida no cadastro (RF01.3).
- **CT18** – Rejeitar confirmação vazia (RF01.4).
- **CT19** – Rejeitar confirmação diferente da senha (RF01.4).
- **CT20** – Aceitar confirmação idêntica (RF01.4).
- **CT21** – Alternância de visibilidade da senha (RF01.4).
- **CT22** – Alternância de visibilidade da confirmação (RF01.4).

8.3 ContagemCortesService (testes de unidade)

- **CT23** – Criar LogPHR corretamente a partir de JSON (RF07).
- **CT24** – Somar corretamente as leituras (RF07).
- **CT25** – Retorna 0 para lista vazia em calcularTotalPecas (RF07).
- **CT26** – Calcular percentual de 50% (RF07).
- **CT27** – Não ultrapassar 100% (RF07).
- **CT28** – Retornar 0% quando total = 0 (RF07).
- **CT29** – Agrupar leituras por hora em gerarSpotsDoGrafico (RF08).
- **CT30** – Retornar lista vazia quando não há logs (RF08).

8.4 AuthServices (testes de integração)

- **CT31** – Criar conta com e-mail único e senha válida → usuário retornado não nulo (RF01.5).
- **CT32** – Realizar login com credenciais recém-criadas → usuário retornado não nulo (RF02.5).
- **CT33** – Tentar login com senha inválida → deve lançar exceção (RF02.6).
- **CT34** – Após criar conta e fazer logout → usuário atual é nulo (RF02.7).
- **CT35** – Chamar reset de senha para e-mail existente → execução sem exceções (RF03.1).

9. Riscos Iniciais

- **R01** – Mensagens de erro dos validadores incorretas ou trocadas (impacta RF01, RF02, RN01).
- **R02** – Validadores podem aceitar entradas inválidas (ex.: e-mail sem “@”, senha com 5 caracteres).
- **R03** – togglePasswordVisibility ou toggleConfirmVisibility podem não notificar listeners, quebrando a atualização da interface.
- **R04** – LogPHR.fromFirestore pode interpretar incorretamente a data/hora.
- **R05** – calcularTotalPecas pode realizar soma errada em listas grandes.
- **R06** – calcularPercentualMeta pode retornar valor > 1.0 ou negativo.
- **R07** – gerarSpotsDoGrafico pode agrupar horas incorretamente ou ignorar logs de minutos diferentes.
- **R08** – Alterações futuras nas classes testadas podem quebrar os testes de unidade existentes.
- **R09** – Falhas de conectividade ou configuração do Firebase podem derrubar os testes de integração (ex.: projeto não inicializado, quota excedida).
- **R10** – O método resetPassword pode falhar silenciosamente, e o teste atual (apenas verifica ausência de exceção) pode não capturar o problema real.
- **R11** – Conflitos de e-mail único (testes simultâneos) podem gerar falhas intermitentes, exigindo uso de timestamps nos e-mails.

10. Critérios de conclusão.

- O escopo dos módulos LoginViewModel, RegisterViewModel, ContagemCortesService e AuthService estiver definido;
- Os requisitos funcionais (RF01, RF02, RF03, RF07, RF08) e a regra de negócio (RN01) pertinentes estiverem identificados e detalhados em sub-requisitos testáveis;
- As condições de teste (CT01 a CT35) estiverem mapeadas para os requisitos e alinhadas com os casos de teste implementados;
- Os riscos principais (R01 a R11) estiverem registrados para monitoramento ao longo do ciclo de testes